

Ch13. 随机算法

Random Algorithm

朱友文

zhuyw@nuaa.edu.cn

随机算法的分类

▶ 随机算法的分类

- 数值计算:
- 蒙特卡罗(Monte Carlo)算法: 一定会返回结果, 但是可能是错的
- 拉斯维加斯(Las Vegas)算法: 返回的结果一定是对的, 但是可能无法返回结果

▶ 随机算法的特点:

- 期望性能优良、适用于大规模数据集
- 简化算法设计, 并降低算法运行开销
- 避免最坏情况
- 结果具有不确定性, 可能错, 可能近似
- 只是期望性能优, 实际执行也可能差
- 需要高质量随机数, 理论分析复杂

随机数发生器

- ▶ 随机数在随机算法设计中扮演着十分重要的角色。在现实计算机上无法产生真正的随机数，因此在随机算法中使用的随机数都是一定程度上随机的，即**伪随机数**。线性同余法是产生伪随机数的最常用的方法。由**线性同余法**产生的随机序列 $a_0, a_1, \dots, a_n$ 满足

$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \bmod m \quad n = 1, 2, \dots \end{cases}$$

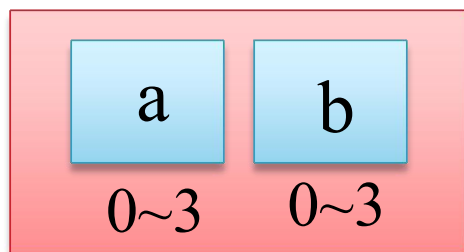
- ▶ 其中 $b \geq 0$, $c \geq 0$, $d \leq m$ 。 d 称为该随机序列的**种子**。如何选取该方法中的常数 **b** 、 **c** 和 **m** 直接关系到所产生的随机序列的随机性能。这是随机性理论研究的内容，已超出本书讨论的范围。
- ▶ 从直观上看， **m** 应取得充分大，因此可取 **m** 为机器大数，另外应取 **$\gcd(m, b) = 1$** ，因此可取 **b** 为一素数。

随机数发生器: 例题01

- ▶ 函数 `Randomt()`; 均匀分布概率随机返回 1到4中的一个整数
- ▶ 如何设计一个算法A, 使得A可以以均匀分布概率返回1到16之间的一个整数?

随机数发生器: 例题01

- ▶ 函数 Randomt(); 均匀分布概率随机返回 1到4中的一个整数
- ▶ 如何设计一个算法A, 使得A可以以均匀分布概率返回1到16之间的一个整数?



$4*a+b$: 0 ~ 15

Random1to16()

```
{  
    a = Randomt()-1;  
    b = Randomt()-1;  
    v = 4*a + b + 1;  
    return v;  
}
```

随机数发生器: 例题02

- ▶ 现有随机数生成函数 `Random01()` 总是以 p 的概率输出0, 以 $1-p$ 的概率输出1, 并且 p 的值未知
- ▶ 如何设计一个算法`RandomB()`, 输出0或1, 并且使得`RandomB()`以0.5的概率返回1, 并以0.5的概率返回0

随机数发生器: 例题02

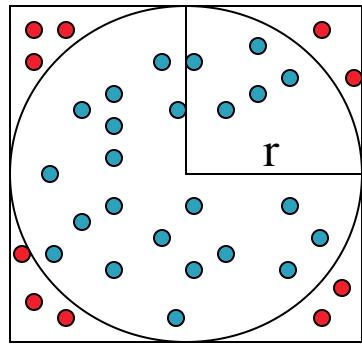
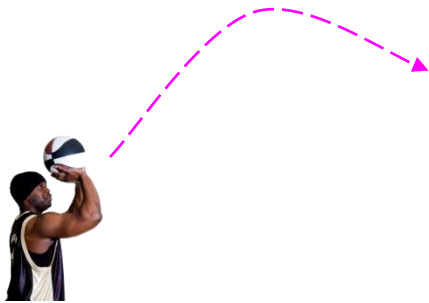
- ▶ 现有随机数生成函数 `Random01()` 总是以 p 的概率输出0, 以 $1-p$ 的概率输出1, 并且 p 的值未知
- ▶ 如何设计一个算法`RandomB()`, 输出0或1, 并且使得`RandomB()`以0.5的概率返回1, 并以0.5的概率返回0

Random01()	Random01()	发生概率
0	0	p^2
0	1	$p(1-p)$
1	0	$p(1-p)$
1	1	$(1-p)^2$

RandomB()

```
{
    while(1){
        x = Random01();
        y = Random01();
        if(x=0 且 y=1) return 0;
        if(x=1 且 y=0) return 1; }
}
```

数值计算：圆周率计算



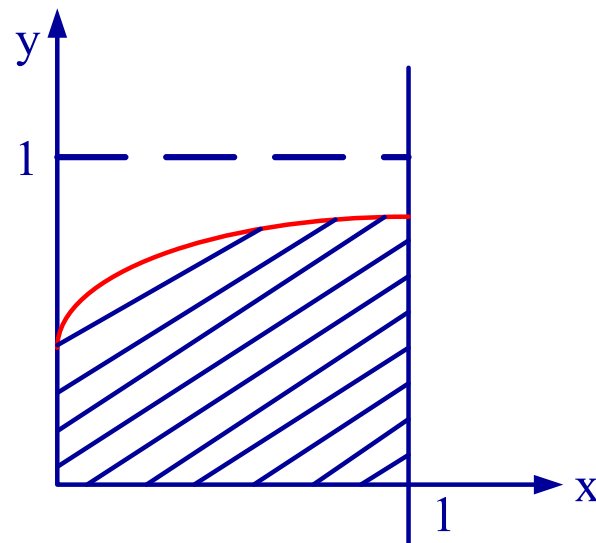
$$\frac{S_{circle}}{S_{square}} = \frac{\pi \cdot r^2}{4 \cdot r^2} \approx \frac{n_{in_circle}}{n_{in_square}} \quad \pi \approx 4 \cdot \frac{n_{in_circle}}{n_{in_square}}$$

```
double Darts(int n)
{ // 用随机投点法计算pi值
  int k=0;
  for (int i=1;i <=n;i++) {
    double x=Random(-1, 1);
    double y=Random(-1, 1);
    if ((x*x+y*y)<=1) k++;
  }
  return 4*k/(double)n;
}
```

一般的数值算法

随机性被最早用于求数值问题的近似解

- 概率算法获得的答案一般是近似的，但通常算法执行的时间越长，精度就越高，误差就越小
- 使用的理由
 - 现实世界中的问题在原理上可能就不存在精确解
例如，无理数，圆周率
 - 精确解存在但无法在可行的时间内求得，有时答案是以置信区间的形式给出的



几个数值计算的例子

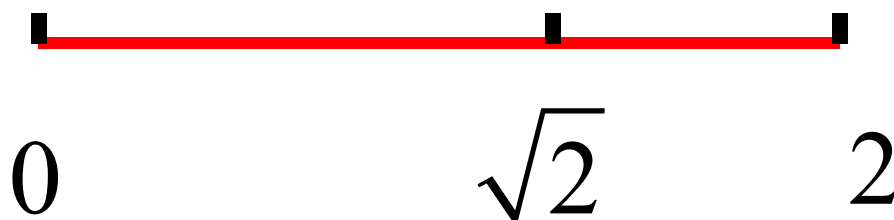
- ▶ 假设已有一个可以生成随机数子函数RandomF(), 其可以均匀随机返回0到2之间的一个随机实数
- ▶ 怎么用随机算法求

$$\sqrt{2}$$

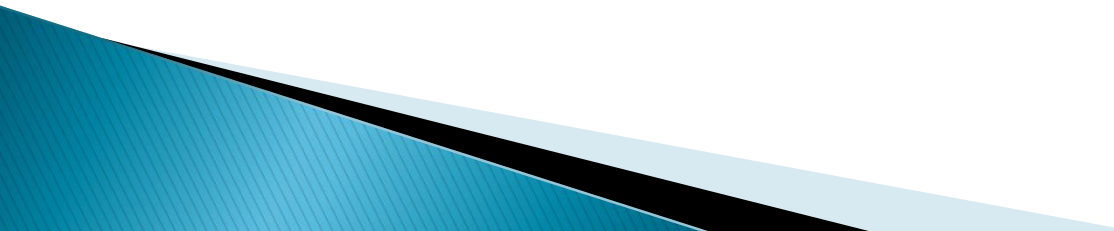
几个数值计算的例子

- ▶ 假设已有一个可以生成随机数子函数RandomF(), 其可以均匀随机返回0到2之间的一个随机实数
- ▶ 怎么用随机算法求

$$\sqrt{2}$$



```
double SquareRoot2(int n)
{
    int k=0;
    for (int i=1;i <=n;i++) {
        double x=RandomF(0, 2);
        if (x*x<=2)  k++;
    }
    return  2*k/(double)n;
}
```



13.3.1 Monte carlo算法举例

- ▶ 特点：总是返回一个答案，但可能是错的
 - 例：A[1, ..., n]是包含n个不同数的数组，n是偶数。返回数组中较大的n/2数中的任一个数x
 - 算法13.1：

```
x = -∞;  
for(i=0; i<k; i++){  
    从数组A中随机选取一个数y;  
    if(y>x) x = y;  
}  
return x;
```

- 正确的概率： $1 - 0.5^k$

13.3.1 Monte carlo算法举例

- ▶ 特点：总是返回一个答案，但可能是错的
- ▶ 素性测试：
 - 最简单的方法
 - 利用 $a^n = a \pmod n$
- ▶ 矩阵相乘结果正确性：习题13.10

设 A, B 和 C 是三个 $n \times n$ 矩阵，写一个 $\Theta(n^2)$ 时间的算法测试是否 $AB = C$ ，如果 $AB = C$ ，算法就返回 *true*。当 $AB \neq C$ 时，算法返回 *true* 的概率有多大？[提示：令 x 是一个 n 个随机向量，测试 $A(Bx) = Cx$ 。]

13.3.2 Las Vegas算法举例

- ▶ 特点：可能无法返回一个答案；但只要返回答案，肯定是对的
- ▶ 例： n 是偶数，数组 $A[1, \dots, n]$ 含有 $(n/2)+1$ 个相同值 x ，另外的元素互异且不等于 x ，设计一个算法找到 x

算法13.2

```
for(t=0; t<k; t++){  
    从1到n中随机选择标号i和j;  
    if(i≠j且A[i]=A[j]) return A[i]; }  
return NULL;
```

13.3.2 Las Vegas算法举例

- ▶ 特点：可能无法返回一个答案；但只要返回答案，肯定是对的
- ▶ 回溯法： n 皇后问题
先随机放置，然后再回溯法处理

13.4 随机化的快速排序

过程 $quicksort(A, low, high)$

1. **if** $low < high$ **then**
2. $SPLIT(A[low \cdots high], w)$ $\{w \text{ 为 } A[low] \text{ 的新位置}\}$
3. $quicksort(A, low, w - 1)$
4. $quicksort(A, w + 1, high)$
5. **end if**

算法 6.5 $SPLIT$

输入：数组 $A[low \cdots high]$ 。

输出：(1) 如有必要，输出按上述描述的重新排列的数组 A ；
(2) 划分元素 $A[low]$ 的新位置 w 。

13.4 随机化的快速排序

▶ 算法

1. $i \leftarrow low$
2. $x \leftarrow A[low]$
3. **for** $j \leftarrow low + 1$ **to** $high$
4. **if** $A[j] \leq x$ **then**
5. $i \leftarrow i + 1$
6. **if** $i \neq j$ **then** 互换 $A[i]$ 和 $A[j]$
7. **end if**
8. **end for**
9. 互换 $A[low]$ 和 $A[i]$
10. $w \leftarrow i$
11. **return** A 和 w

13.4 随机化的快速排序

▶ 算法

过程 $rquicksort(low, high)$

1. **if** $low < high$ **then**
2. $v \leftarrow random(low, high)$
3. 互换 $A[low]$ 和 $A[v]$
4. $SPLIT(A[low \cdots high], w)$ $\{w \text{ 是主元的新位置}\}$
5. $rquicksort(low, w - 1)$
6. $rquicksort(w + 1, high)$
7. **end if**

▶ 期望时间

5.5 寻找第 k 小元素

➤ 解决思路:

(1) 找出一个候选值 mm

令 $q = \lfloor p/5 \rfloor$ 。将 A 分成 q 组，每组 5 个元素。如果 5 不整除 p ，则排除剩余的元素。

将 q 组中的每一组单独排序，找出中项。所有中项的集合为 M 。

然后，选出 q 个中项的中项 足第 k 小元素的条件，
如果否，继续在下一个候选值进行上述操作： 三种情况

$$|A_1| \quad |A_1| + |A_2| < k \quad |A_2| \geq k$$

13.5 随机化的选择算法

▶ 算法 $rselect(A, low, high, k)$

1. $v \leftarrow random(low, high)$

2. $x \leftarrow A[v]$

3. 将 $A[low \cdots high]$ 分成三个数组

$$A_1 = \{a \mid a < x\}$$

$$A_2 = \{a \mid a = x\}$$

$$A_3 = \{a \mid a > x\}$$

4. **case**

$|A_1| \geq k$: **return** $rselect(A_1, 1, |A_1|, k)$

$|A_1| + |A_2| \geq k$: **return** x

$|A_1| + |A_2| < k$: **return** $rselect(A_3, 1, |A_3|, k - |A_1| - |A_2|)$

5. **end case**

13.5 随机化的选择算法

▶ 期望时间

$$C(n) = n + \frac{1}{n} \left[\sum_{j=1}^{k-1} C(n-j) + \sum_{j=k+1}^n C(j-1) \right]$$

$$= n + \frac{1}{n} \left[\sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j) \right]$$

13.5 随机化的选择算法

▶ 期望时间

$$C(n) = n + \frac{1}{n} \left[\sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j) \right]$$

$C(n-1)$	$C(n-1)$
$C(n-2)$	$C(n-2)$
$\cdot$	$\cdot$
$\cdot$	$\cdot$
$\cdot$	$\cdot$
$C(n-k+1)$	$C(k)$

13.5 随机化的选择算法

▶ 期望时间

$$\begin{aligned} C(n) &= n + \frac{1}{n} \left[\sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j) \right] \\ &\leq n + \frac{1}{n} \left[\sum_{j=n-\lceil n/2 \rceil+1}^{n-1} C(j) + \sum_{j=\lceil n/2 \rceil}^{n-1} C(j) \right] \end{aligned}$$

▶ 进一步，归纳法可证

$$C(n) \leq 4n$$